

ESP32 System Firmware Source Code (system_firmware.cpp)

Anti-Wall-Jumping Perimeter Security System

```
/*
 * STUDENT VERSION - RFID Anti-Wall-Jumping System
 *
 * This firmware reads RFID tags, detects wall climbing via an IR beam sensor,
 * and sends an SMS alert via a SIM800L module if an intrusion is detected.
 */

#include <DFRobotDFPlayerMini.h> // For playing alarm sounds and voice messages
#include <HardwareSerial.h>       // For communicating with SIM800L and DFPlayer
#include <Wiegand.h>              // For reading the RFID tags from the Wiegand reader
#include <SD.h>                   // For saving logs and settings to SD card
#include <SPI.h>
#include <Wire.h>
#include <vector>

// --- PIN DEFINITIONS ---
#define WG_D0_PIN    21 // Wiegand Data 0
#define WG_D1_PIN    22 // Wiegand Data 1
#define SD_CS_PIN     5 // SD Card Chip Select

#define RTC_SCL_PIN   26 // Real Time Clock SCL
#define RTC_SDA_PIN   27 // Real Time Clock SDA

#define SIM800_TX_PIN 17 // SIM800L TX -> ESP32 RX
#define SIM800_RX_PIN 16 // SIM800L RX -> ESP32 TX

#define DFPLAYER_TX_PIN 2 // DFPlayer TX -> ESP32 RX
#define DFPLAYER_RX_PIN 15 // DFPlayer RX -> ESP32 TX

#define ALARM_LED_PIN 4 // Pin for the visual alarm LED
#define STATUS_LED_PIN 13 // Status indicator LED
#define MODE_BUTTON_PIN 34 // Button to switch between normal and configuration modes

#define IR_BEAM_PIN 14 // Infrared beam sensor to detect wall climbing
#define HOLD_TIME_MS 3000 // Time required to hold the mode button (3 seconds)

// --- SYSTEM VARIABLES ---
// Mutexes are used to prevent multiple tasks from accessing the same resource at the same time
inline SemaphoreHandle_t spiMutex = nullptr;
inline SemaphoreHandle_t configMutex = nullptr;
inline SemaphoreHandle_t dbMutex = nullptr;
inline SemaphoreHandle_t scanMutex = nullptr;

inline bool sdAvailable = false;
inline bool dfPlayerAvailable = false;

// We include custom modules for Real Time Clock, Storage and Web configuration
#include "rtc_module.h"
#include "storage.h"
#include "web_portal.h" // (Included for config portal methods used below)

SystemConfig sysConfig; // Stores the current system configuration

volatile bool scanModeActive = false;
char lastScannedTag[32] = {0};
String simStatus = "Initializing...";
String simNumber = "Unknown";
std::vector<String> recentTags; // Stores recently scanned RFID tags
unsigned long lastTagTime = 0;

// Hardware interfaces
WIEGAND wg;
HardwareSerial sim800(2);
HardwareSerial dfSerial(1);
DFRobotDFPlayerMini myDFPlayer;
```

```

enum SystemMode { MODE_NORMAL, MODE_CONFIG };
volatile SystemMode currentMode = MODE_NORMAL; // System starts in normal operating mode
volatile bool requestConfigMode = false;
volatile bool requestNormalMode = false;
unsigned long lastInteractionTime = 0;

// Data structure to hold an alarm event for SMS
struct AlarmEvent {
    char names[512]; // Names of students caught
    int count;       // Number of students
};

QueueHandle_t alarmQueue; // Queue to pass alarm events to the SMS task
volatile bool alarmActive = false;
unsigned long alarmStartTime = 0;
unsigned long lastAlarmTime = 0;

// --- FUNCTION PROTOTYPES ---
void sensorTask(void *pvParameters);
void commTask(void *pvParameters);
void startLocalAlarm(int voiceTrack);
void updateLocalAlarm();
bool checkIRBeam();
String readRFID();
void enterConfigMode();
void exitConfigMode();
void updateStatusLED();
void checkSIM800Ready();
void flushSim800Response();
void testPlayVoice(int track);
void stopVoice();

void setup() {
    Serial.begin(115200);
    delay(200);
    Serial.println("\n[BOOT] Anti-Wall-Jumping System starting...");

    // Create mutexes for thread-safe operations
    spiMutex = xSemaphoreCreateMutex();
    configMutex = xSemaphoreCreateMutex();
    dbMutex = xSemaphoreCreateMutex();
    scanMutex = xSemaphoreCreateMutex();

    // Load configuration from EEPROM (internal memory)
    EEPROM.begin(sizeof(EEPROMConfig));
    loadConfigFromEEPROM(sysConfig);

    // Set pin modes
    pinMode(MODE_BUTTON_PIN, INPUT);
    pinMode(STATUS_LED_PIN, OUTPUT);
    pinMode(IR_BEAM_PIN, INPUT_PULLUP); // Use internal pull-up resistor for IR beam
    pinMode(ALARM_LED_PIN, OUTPUT);

    digitalWrite(ALARM_LED_PIN, LOW);
    digitalWrite(STATUS_LED_PIN, HIGH);

    // Initialize the Wiegand RFID reader
    wg.begin(WG_D0_PIN, WG_D1_PIN);
    Serial.println("[BOOT] Wiegand Reader initialized.");

    // Initialize the SD card securely using the SPI mutex
    if (spiMutex && xSemaphoreTake(spiMutex, portMAX_DELAY) == pdTRUE) {
        bool sdOk = SD.begin(SD_CS_PIN);
        if (!sdOk) {
            Serial.println("[WARN] SD card missing. Running with EEPROM configuration.");
            sdAvailable = false;
        } else {
            Serial.println("[BOOT] SD card ready");
            sdAvailable = true;
        }

        // Load configuration and student database from SD if available
        if (SD.exists(CONFIG_FILE_PATH)) {
            loadConfigFromSD(sysConfig);
        }
    }
}

```

```

    } else {
        saveConfigToSD(sysConfig);
    }
    loadStudentsFromSD();
}
xSemaphoreGive(spiMutex);
}

// Initialize Real Time Clock
Wire.begin(RTC_SDA_PIN, RTC_SCL_PIN);
initRTC();

// Initialize DFPlayer Mini for playing voice alarms
dfSerial.begin(9600, SERIAL_8N1, DFPLAYER_RX_PIN, DFPLAYER_TX_PIN);
if (myDFPlayer.begin(dfSerial)) {
    Serial.println("[BOOT] DFPlayer Mini ready.");
    myDFPlayer.volume(30);
    dfPlayerAvailable = true;
}

// Initialize SIM800L for SMS capabilities
sim800.begin(9600, SERIAL_8N1, SIM800_RX_PIN, SIM800_TX_PIN);
Serial.println("[BOOT] Initializing SIM800L...");
checkSIM800Ready();

// Create the FreeRTOS queue for alarms
alarmQueue = xQueueCreate(20, sizeof(AlarmEvent));

// Start the background tasks: one for sensors, one for SMS communication
// This allows the ESP32 to monitor sensors and send SMS at the same time!
xTaskCreatePinnedToCore(sensorTask, "SensorTask", 8192, NULL, 1, NULL, 0);
xTaskCreatePinnedToCore(commTask, "CommTask", 8192, NULL, 1, NULL, 1);

Serial.println("[BOOT] System Ready.");
}

void loop() {
    // --- BUTTON LOGIC ---
    // The loop handles checking the physical mode button (Configuration vs Normal mode)
    static unsigned long pressStart = 0;
    static bool pressing = false;
    static bool actionFired = false;
    static unsigned long lastDebounceTime = 0;
    static bool lastButtonState = HIGH;

    bool reading = digitalRead(MODE_BUTTON_PIN);
    if (reading != lastButtonState) {
        lastDebounceTime = millis();
    }

    bool pressed = false;
    if ((millis() - lastDebounceTime) > 50) { // Debounce button
        pressed = (reading == LOW);
    }
    lastButtonState = reading;

    if (pressed && !pressing) {
        pressing = true;
        actionFired = false;
        pressStart = millis();
    } else if (!pressed && pressing) {
        pressing = false;
    }

    // If button is held for 3 seconds, toggle modes
    if (pressing && !actionFired && (millis() - pressStart >= HOLD_TIME_MS)) {
        actionFired = true;
        if (currentMode == MODE_NORMAL) enterConfigMode();
        else exitConfigMode();
    }

    // Switch modes based on SMS commands (handled in commTask)
    if (requestConfigMode) {

```

```

    requestConfigMode = false;
    if (currentMode == MODE_NORMAL) enterConfigMode();
}
if (requestNormalMode) {
    requestNormalMode = false;
    if (currentMode == MODE_CONFIG) exitConfigMode();
}

// Service the Web Captive Portal if in Config Mode
if (currentMode == MODE_CONFIG) {
    serviceCaptivePortal();
    updateStatusLED();
    vTaskDelay(pdMS_TO_TICKS(2));
} else {
    updateStatusLED();
    vTaskDelay(pdMS_TO_TICKS(50));
}
}

// --- MODE SWITCHING ---
void enterConfigMode() {
    currentMode = MODE_CONFIG;
    alarmActive = false;
    if (dfPlayerAvailable) myDFPlayer.stop();
    digitalWrite(ALARM_LED_PIN, LOW);
    startCaptivePortal(); // Turn on WiFi AP for web configuration
}

void exitConfigMode() {
    stopCaptivePortal(); // Turn off WiFi AP
    currentMode = MODE_NORMAL;
}

// Blinks the LED quickly during config mode, solid during normal mode
void updateStatusLED() {
    static unsigned long lastBlink = 0;
    if (currentMode == MODE_CONFIG) {
        if (millis() - lastBlink > 400) {
            digitalWrite(STATUS_LED_PIN, !digitalRead(STATUS_LED_PIN));
            lastBlink = millis();
        }
    } else {
        digitalWrite(STATUS_LED_PIN, HIGH);
    }
}

// --- SENSOR TASK (Core 0) ---
// This task continuously checks for RFID tags and IR beam breaks
void sensorTask(void *pvParameters) {
    unsigned long tagSeenTime = 0;
    const unsigned long TAG_MEMORY_MS = 5000; // How long a scanned tag is remembered

    for (;;) {
        // 1. Check for RFID tag
        String tagID = readRFID();
        if (tagID != "") {
            tagSeenTime = millis();

            if (scanMutex && xSemaphoreTake(scanMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
                bool exists = false;
                for (const String& t : recentTags) {
                    if (t == tagID) exists = true;
                }
                // Save the tag if not already in the recent list
                if (!exists) {
                    recentTags.push_back(tagID);
                    Serial.println("[SENSOR] Captured tag into buffer: " + tagID);
                }
                lastTagTime = millis();
                strncpy((char*)lastScannedTag, tagID.c_str(), sizeof(lastScannedTag) - 1);
                xSemaphoreGive(scanMutex);
            }
        }
    }
}

```

```

// Forget tags after 5 seconds of inactivity
if (scanMutex && xSemaphoreTake(scanMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
    if (!recentTags.empty() && (millis() - lastTagTime > TAG_MEMORY_MS)) {
        recentTags.clear();
        lastScannedTag[0] = '\0';
    }
    xSemaphoreGive(scanMutex);
}

// Don't trigger alarms while configuring the system
if (currentMode == MODE_CONFIG) {
    vTaskDelay(pdMS_TO_TICKS(10));
    continue;
}

// Fetch system configurations
unsigned long cooldown = 10000; // Minimum time between SMS alarms
String wall = "Wall";
int voiceTrack = 1;

if (configMutex && xSemaphoreTake(configMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
    cooldown = sysConfig.cooldown;
    wall = sysConfig.wallLabel;
    voiceTrack = sysConfig.alarmVoice;
    xSemaphoreGive(configMutex);
}

// 2. Check for Intrusion (IR Beam Break)
bool climbingDetected = checkIRBeam();
static unsigned long lastBreachTime = 0;
const unsigned long BREACH_MEMORY_MS = 3000; // Remember the breach for 3 seconds

if (climbingDetected) {
    lastBreachTime = millis();
}

bool recentBreach = (lastBreachTime > 0) && (millis() - lastBreachTime <= BREACH_MEMORY_MS);

// 3. Trigger Alarm if climbing is detected AND an RFID tag was recently read
if (recentBreach && (millis() - lastAlarmTime > cooldown)) {
    bool hasTags = false;
    std::vector<String> tagsCopy;
    if (scanMutex && xSemaphoreTake(scanMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
        if (!recentTags.empty()) {
            hasTags = true;
            tagsCopy = recentTags; // Copy tags for processing
        }
        xSemaphoreGive(scanMutex);
    }

    if (hasTags) {
        // Intrusion confirmed! Start the local siren/voice alarm
        startLocalAlarm(voiceTrack);
        lastAlarmTime = millis();

        // Prepare the SMS message with the student names
        AlarmEvent event;
        event.count = tagsCopy.size();
        String nameList = "";

        std::vector<BatchLogEntry> logBatch;

        // Lookup each student's name from their RFID
        for (const String& t : tagsCopy) {
            String s = lookupStudentName(t);
            String st_id = lookupStudentID(t);

            if (s == "") {
                s = "Unregistered";
                st_id = t.substring(0, 4);
            }
            logBatch.push_back({t, s});
        }
    }
}

```

```

        String bulletLine = "\n- (" + st_id + ") " + s;
        if (nameList.length() + bulletLine.length() + 20 < 500) {
            nameList += bulletLine;
        }
    }

    // Save to SD card Log
    logMultipleEventsToSD(logBatch, wall, "1");
    nameList.toCharArray(event.names, sizeof(event.names));

    // Send the event to the communication task to dispatch the SMS
    xQueueSend(alarmQueue, &event, 0);

    // Clear the tag buffer and reset breach
    if (scanMutex && xSemaphoreTake(scanMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
        recentTags.clear();
        xSemaphoreGive(scanMutex);
    }
    lastBreachTime = 0;
}

// Check if the local alarm should be turned off automatically
updateLocalAlarm();
vTaskDelay(pdMS_TO_TICKS(10)); // Yield to other tasks
}
}

struct SMSJob {
    bool active = false;
    String namesList;
    int studentCount = 0;
    String phone;
    String wall;
    String role;
    int state = 0;
    unsigned long stateTime = 0;
};

// --- COMMUNICATION TASK (Core 1) ---
// This task handles sending and receiving SMS via the SIM800L
void commTask(void *pvParameters) {
    AlarmEvent event;
    String incomingSMS = "";
    SMSJob currentSMS;

    for (;;) {
        // 1. Process outgoing SMS alarms
        if (currentMode != MODE_CONFIG) {
            // If there is an alarm in the queue, prepare to send it
            if (!currentSMS.active && xQueueReceive(alarmQueue, &event, pdMS_TO_TICKS(50)) == pdTRUE) {
                currentSMS.namesList = String(event.names);
                currentSMS.studentCount = event.count;
                if (configMutex && xSemaphoreTake(configMutex, portMAX_DELAY) == pdTRUE) {
                    currentSMS.phone = sysConfig.guardPhone;
                    currentSMS.wall = sysConfig.wallLabel;
                    currentSMS.role = sysConfig.recipientRole;
                    xSemaphoreGive(configMutex);
                }
                currentSMS.active = true;
                currentSMS.state = 0; // SMS sending state machine
                currentSMS.stateTime = millis();
            }
            else {
                vTaskDelay(pdMS_TO_TICKS(50));
            }
        }

        // State machine to send SMS safely without blocking the program
        if (currentSMS.active) {
            unsigned long now = millis();
            switch (currentSMS.state) {
                case 0:

```

```

sim800.println("AT+CMGF=1"); // Set SMS to text mode
currentSMS.state = 1;
currentSMS.stateTime = now;
break;
case 1:
    if (now - currentSMS.stateTime >= 200) {
        sim800.print("AT+CMGS=\""); // Prepare phone number
        sim800.print(currentSMS.phone);
        sim800.println("\");
        currentSMS.state = 2;
        currentSMS.stateTime = now;
    }
    break;
case 2:
    if (now - currentSMS.stateTime >= 200) {
        // Write the actual SMS content
        sim800.print(currentSMS.role == "Teacher" ? "[CAMPUS ALERT - FACULTY] " : "[CAMPUS ALERT - SECURITY]
");
        sim800.print("ALERT: ");
        sim800.print(currentSMS.studentCount);
        sim800.print(" Student(s) caught climbing over ");
        sim800.print(currentSMS.wall);
        sim800.print("! IDs: ");
        sim800.print(currentSMS.namesList);
        currentSMS.state = 3;
        currentSMS.stateTime = now;
    }
    break;
case 3:
    if (now - currentSMS.stateTime >= 100) {
        sim800.write(26); // Send CTRL+Z to send the SMS
        currentSMS.state = 4;
        currentSMS.stateTime = now;
    }
    break;
case 4:
    if (now - currentSMS.stateTime >= 3000) {
        currentSMS.active = false; // SMS completed
    }
    break;
}
}

// 2. Process incoming SMS commands
int maxBytes = 128;
while (sim800.available() && maxBytes-- > 0) {
    char c = sim800.read();
    incomingSMS += c;
}

if (incomingSMS.length() > 0) {
    // Prevent buffer overflow
    if (incomingSMS.length() > 512) {
        incomingSMS = incomingSMS.substring(256);
    }

    String upperSMS = incomingSMS;
    upperSMS.toUpperCase();

    // Look for specific commands in the message
    int idxOn = upperSMS.indexOf("AP ON");
    int idxOff = upperSMS.indexOf("AP OFF");
    int idxReset = upperSMS.indexOf("RESTORE FACTORY SETTINGS");

    if (idxOn != -1 || idxOff != -1 || idxReset != -1) {

        // Parse the sender's phone number
        String senderNum = "";
        int cmtIdx = upperSMS.lastIndexOf("+CMT:");
        if (cmtIdx != -1) {
            int q1 = incomingSMS.indexOf('\'', cmtIdx);
            int q2 = incomingSMS.indexOf('\'', q1 + 1);
            if (q1 != -1 && q2 != -1) {

```

```

        senderNum = incomingSMS.substring(q1 + 1, q2);
    }
}

int minIdx = 9999;
int cmdLen = 0;
int activeCmd = 0; // 3=Reset, 4=Off, 5=On

if (idxReset != -1 && idxReset < minIdx) { minIdx = idxReset; cmdLen = 24; activeCmd = 3; }
if (idxOff != -1 && idxOff < minIdx) { minIdx = idxOff; cmdLen = 6; activeCmd = 4; }
if (idxOn != -1 && idxOn < minIdx) { minIdx = idxOn; cmdLen = 5; activeCmd = 5; }

// Process authorized commands (e.g., from the guard's phone)
if (activeCmd == 3 || activeCmd == 4 || activeCmd == 5) {
    bool isGuard = false;

    if (configMutex && xSemaphoreTake(configMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
        String rawPhone = sysConfig.guardPhone;
        String digits = "";
        for (int i = 0; i < rawPhone.length(); i++) {
            if (isDigit(rawPhone[i])) digits += rawPhone[i];
        }
        if (digits.length() > 0 && senderNum.indexOf(digits) != -1) {
            isGuard = true;
        }
        xSemaphoreGive(configMutex);
    }

    if (activeCmd == 5 && isGuard) {
        requestConfigMode = true; // Turn ON WiFi Portal
    } else if (activeCmd == 4 && isGuard) {
        requestNormalMode = true; // Turn OFF WiFi Portal
    } else if (activeCmd == 3 && isGuard) {
        extern void performFactoryReset();
        performFactoryReset();
    }

    // Remove processed command from buffer
    incomingSMS = incomingSMS.substring(minIdx + cmdLen);
    continue;
}
}
}
vTaskDelay(pdMS_TO_TICKS(10));
}
}

// --- HELPER FUNCTIONS ---

// Checks if the IR Beam is broken (climbing detected)
bool checkIRBeam() {
    return digitalRead(IR_BEAM_PIN) == HIGH;
}

// Reads the RFID tag from the Wiegand interface
String readRFID() {
    String tag = "";
    if (wg.available()) {
        unsigned long code = wg.getCode();
        tag = String(code, HEX);
        tag.toUpperCase();
        Serial.println("[RFID] Scanned Tag: " + tag);
    }
    return tag;
}

// Turns on the siren/voice and the visual LED alarm
void startLocalAlarm(int voiceTrack) {
    alarmActive = true;
    alarmStartTime = millis();
    if (dfPlayerAvailable) myDFPlayer.play(voiceTrack);
    digitalWrite(ALARM_LED_PIN, HIGH);
}

```



```

// Automatically turns off the alarm after the configured duration
void updateLocalAlarm() {
    unsigned long duration = 5000;
    if (configMutex && xSemaphoreTake(configMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
        duration = sysConfig.alarmDuration;
        xSemaphoreGive(configMutex);
    }
    if (alarmActive && (millis() - alarmStartTime >= duration)) {
        alarmActive = false;
        if (dfPlayerAvailable) myDFPlayer.stop();
        digitalWrite(ALARM_LED_PIN, LOW);
    }
}

// Initializes SIM800L and waits for GSM Network Registration
void checkSIM800Ready() {
    sim800.println("AT");
    delay(500);
    flushSim800Response();

    unsigned long startWait = millis();
    while (millis() - startWait < 60000) {
        sim800.println("AT+CREG?"); // Check registration status
        delay(1000);

        String response = "";
        while (sim800.available()) { response += (char)sim800.read(); }

        // "0,1" or "0,5" means successfully registered on home or roaming network
        if (response.indexOf("+CREG: 0,1") != -1 || response.indexOf("+CREG: 0,5") != -1) {
            break;
        }
        delay(1000);
    }

    sim800.println("AT+CMGF=1"); // Set SMS to Text Mode
    delay(200);
    sim800.println("AT+CNMI=1,2,0,0,0"); // Route incoming SMS directly to Serial port
    delay(200);
    flushSim800Response();
}

void flushSim800Response() {
    while (sim800.available()) {
        Serial.write(sim800.read());
    }
}

void testPlayVoice(int track) { if (dfPlayerAvailable) myDFPlayer.play(track); }
void stopVoice() { if (dfPlayerAvailable) myDFPlayer.stop(); }

```